Safe Projection onto Linear Safety Constraints

SafeGen-SBP: Engineering Audit, Geometric Foundations, and Numerical Verification (v2.2.0)

Author: Mobin Khorshidvand

Final verification environment: Python 3.12.3 · NumPy 2.1.3 · SciPy 1.14.1

Abstract

SafeGen-SBP (v2.2.0) provides a practical Python implementation for computing Euclidean projections onto linear half-spaces [1]. The package combines the closed-form single-constraint projection formula with Dykstra's cyclic projection algorithm for intersecting constraints [2], supplemented by an independent feasibility check and benchmark comparisons against SciPy's SLSQP solver. Rather than introducing a novel optimization method or claiming long-term closed-loop safety guarantees, this work focuses on numerical reliability, edge-case auditing, and reproducible verification. All eight unit and integration tests passed during our final verification run. Because our latest benchmarks differed from several figures noted in an earlier draft—specifically regarding raw violation rates and execution speed—this report presents the updated measurements taken directly from the verified execution logs.

Keywords: Euclidean projection; linear constraints; Dykstra's algorithm; numerical verification; reproducibility.

1 Problem Definition and Geometric Motivation

In many control and machine learning applications, continuous outputs generated by a model may occasionally violate operational linear inequalities. A straightforward way to enforce these bounds is to apply a post-processing filter that maps any offending vector to the nearest point within the safe set [1].

For a single linear inequality, we define the closed half-space as $H = \{x \in \mathbb{R}^n \mid g^T x \leq \beta\}$, where $g \neq 0$. The orthogonal (Euclidean) projection of a point $x \in \mathbb{R}^n$ onto H is given in closed form by [1]:

$$P_H(x) = x - \frac{\max(0, g^T x - \beta)}{g^T g} g.$$

Geometrically, if the input vector already satisfies the constraint ($g^T x \leq \beta$), the filter leaves it untouched. When a violation occurs, the formula subtracts a scalar multiple of the normal vector g, pulling the point orthogonally back onto the boundary hyperplane $g^T x = \beta$. This operator is *piecewise affine*—acting as the identity map inside the feasible region and an affine projection outside—and remains continuous across the boundary, though it is generally non-differentiable where $g^T x = \beta$. Note that this operation stops at the boundary rather than reflecting the vector across it.

1.1 Two-Dimensional Walkthrough

To see how this works in practice, consider a simple two-dimensional case with input $x = (4, 5)$, normal vector $g = (1, 1)$, and bound $\beta = 4$. Evaluating the inner product gives $g^T x = 9$, which exceeds β by a violation of $9 - 4 = 5$. Since the squared norm of the normal vector is $g^T g = 2$, the projection step scales g by $5/2 = 2.5$:

$$P_H(x) = (4, 5) - 2.5(1, 1) = (1.5, 2.5).$$

The resulting coordinates sum to $1.5 + 2.5 = 4$, placing the point squarely on the constraint boundary. We include this calculation as Test 1 in our verification suite. Keep in mind that this projection is strictly a pointwise geometric correction at the current step and makes no assumptions about how the underlying system will evolve over subsequent time steps.

1.2 Scope and Contribution

Both half-space projection [1] and Dykstra's algorithm [2] are well-established tools in convex analysis, so we make no claims of mathematical novelty here. Instead, the contribution of this work lies in providing a clean, well-documented NumPy implementation, auditing the numerical pitfalls of adding small regularization offsets to the denominator, and supplying a fully reproducible suite of unit tests and solver benchmarks.

2 Numerical Policy and Multiple Constraints

2.1 Squared-Normal Cutoff

When implementing projection formulas in floating-point arithmetic, nearly zero normal vectors can cause severe numerical instability. To guard against division by underflowed values, our implementation enforces a strict cutoff of $g^T g \leq 10^{-14}$ on the *squared* Euclidean norm. We treat this threshold as a practical engineering safeguard rather than a mathematical redefinition of zero.

Under this rule, a normal vector with $\|g\|_2 = 10^{-6}$ yields $g^T g = 10^{-12}$ and is processed normally, whereas a vector with $\|g\|_2 = 10^{-8}$ gives $g^T g = 10^{-16}$ and is rejected as degenerate. Exact zeros, non-finite inputs (`NaN` or `Inf`), incompatible dimensions, and invalid tolerances are similarly caught and raised as `ValueError` exceptions (verified in Test 3 and Test 4). Because multiplying a constraint inequality by a positive scalar changes the magnitude of $g^T g$, users should be mindful of how their constraints are scaled before passing them to the filter.

2.2 Effect of Adding 10^{-12} to the Denominator

A common habit in numerical coding is to tack a small constant, such as $\epsilon = 10^{-12}$, onto the denominator $g^T g + \epsilon$ to prevent division by zero. While well-intentioned, this additive offset silently corrupts the projection whenever the constraint normal happens to be small.

For instance, take $g = (10^{-6}, 0)$, $x = (2, 5)$, and $\beta = 10^{-6}$. Here, the squared norm is $g^T g = 10^{-12}$ and the constraint violation is $g^T x - \beta = 10^{-6}$. Using the exact formula, the scaling multiplier is $10^{-6}/10^{-12} = 10^6$, which shifts x to the true boundary point $(1, 5)$. However, if 10^{-12} is added to the denominator, the divisor doubles to 2×10^{-12}. This cuts the corrective step in half and yields $(1.5, 5)$—a point that still violates the safety constraint.

We verify this exact scenario in Test 4. Note that the helper function `legacy_project_buggy` included in our codebase is simply a pedagogical reconstruction designed to illustrate this specific numerical flaw; it is not meant to imply that any particular third-party library contains this bug.

2.3 Dykstra's Method and Status Reporting

When dealing with a polyhedral intersection of multiple constraints $Gx \leq b$, simply cycling through individual half-space projections (Von Neumann's alternating projections) generally converges to a feasible point, but not necessarily the closest one in Euclidean distance. To find the true metric projection onto the intersection, we implement Dykstra's cyclic projection algorithm [2]. Dykstra's method maintains an auxiliary correction vector for each constraint: before projecting onto a given half-space, the algorithm adds that constraint's previous correction back to the current iterate, performs the single-half-space projection, and updates the stored correction as the difference.

In our implementation, we carefully separate convergence criteria—such as step-size tolerances (`step_tol`) and iteration limits (`max_iters`)—from actual constraint satisfaction (`is_feasible`). A solution is flagged as certified feasible (`is_feasible=True`, returning `x`) only after an independent post-hoc check confirms that the maximum signed constraint violation across all rows of $Gx \leq b$ falls at or below `feas_tol`.

This distinction is crucial in practice: exhausting the iteration budget (`budget_exhausted=True`) does not prove that a problem is infeasible, just as a tiny step between successive iterates does not guarantee that the current point is safe. For example, when fed contradictory constraints ($x_1 + x_2 \leq 2$ and $-x_1 - x_2 \leq -5$) in Test 7, the solver hits its 50-cycle limit with a maximum signed violation of `3.0` and properly reports a failure to certify a feasible point (`x=None`), without falsely claiming a formal certificate of infeasibility.

3 Experimental Setup and Results

All empirical results reported below were generated in our final verification run (recorded in `results/final_verification.log`, or `../results/final_verification.log` relative to `docs/`) using Python 3.12.3, NumPy 2.1.3, and SciPy 1.14.1. To ensure consistency, all random sampling was seeded across five fixed runs (`seeds = [42, 43, 44, 45, 46]`) via `numpy.random.default_rng(seed)`.

3.1 Benchmark 1: Single Half-Space

In our first benchmark, we tested projection in $\mathbb{R}^{50}$ using a single random constraint direction per seed, normalized to $\|g\|_2 = 10^{-6}$ with bound $\beta = 0$. For each of the five seeds, we drew 1,000 sample vectors from a normal distribution $\mathcal{N}(0.5, 1.0)$, totaling 5,000 test points. Identical inputs were passed to the unfiltered baseline (Raw), the denominator-offset model (Reconstructed $+10^{-12}$), and our exact projection implementation. Table 1 summarizes the mean and population standard deviation of the five per-seed violation rates, alongside the mean of the five per-seed maximum violations.

Table 1: Single half-space projection performance across 5,000 samples (5 seeds $\times$ 1,000 inputs).

Method	**Violation Rate (Mean $\pm$ SD)**	**Mean of Per-Seed Max Violation**
Raw (Unfiltered)	`52.200000%` $\pm$ `20.801250%`	`3.334013636005e-06`
Reconstructed ($+10^{-12}$ offset)	`52.200000%` $\pm$ `20.801250%`	`1.667006818003e-06`
Exact projection	`0.000000%` $\pm$ `0.000000%`	`1.072201025912e-21`

To be fully transparent about how violations were tallied in the code and log: the raw baseline flagged any strictly positive residual ($g^T x - \beta > 0$), the reconstructed-offset test used a threshold of 10^{-15}, and the exact projection check counted residuals exceeding 10^{-12}. Because these thresholds differ slightly, they are explicitly disclosed; however, even if evaluated against the strictest threshold (10^{-15}), the exact method's worst-case per-seed residual averaged `1.072201025912e-21`, which is zero up to floating-point rounding.

As seen in Table 1, adding the 10^{-12} offset cuts the magnitude of the violation in half (leaving a mean per-seed maximum Euclidean gap of `1.667006818003` between the offset and exact outputs) but fails to bring the violating points across the boundary. We also note that a raw violation rate of `74.72%` $\pm$ `0.83%` mentioned in an earlier unverified draft could not be reproduced under this seeding setup (where one random direction g is drawn per seed) and has been discarded in favor of the logged figures above.

3.2 Benchmark 2: Polytope Projection vs. SLSQP

Our second benchmark evaluated multi-constraint projection in $\mathbb{R}^{20}$ subject to 21 linear inequalities: an upper bound on the sum of coordinates ($\sum_{i=1}^{20} x_i \leq 15$) alongside twenty nonnegativity bounds ($-x_i \leq 0$, i.e., $x_i \geq 0$). Across five seeds, we generated 200 points per seed (1,000 paired inputs overall) from $\mathcal{N}(1, 2)$. We configured Dykstra's algorithm with `step_tol = 1e-8`, `feas_tol = 1e-7`, and `max_iters = 500`. For comparison, we solved the equivalent quadratic program $\min_y \frac{1}{2}\|y - x\|_2^2$ subject to $Gy \leq b$ using SciPy's SLSQP routine with `ftol = 1e-12` and `maxiter = 500`. Success counters for Dykstra and SLSQP are tracked independently in the benchmark loop.

Across all 1,000 test instances, Dykstra's method converged to a certified feasible solution every time (`1000/1000`), requiring an average of `27.945000` cycles and achieving an overall maximum constraint violation of `9.999362582391e-08` (within the 10^{-7} target tolerance). SciPy's SLSQP solver likewise succeeded on all `1000/1000` problems with a maximum violation of `1.044497821567e-12`. Comparing the solutions returned by both solvers across all 1,000 jointly successful cases, the maximum L_∞ difference was only `1.632513857430e-08`, confirming that both methods arrive at virtually identical projections.

4 Timing, Discussion, and Limitations

During the Benchmark 2 run, we recorded wall-clock execution times sequentially using Python's `time.perf_counter`, measuring each sample in a single pass without warm-up passes or multi-trial averaging. In our test environment, Dykstra's algorithm averaged `13.620408 ms` per sample, whereas SciPy's SLSQP took only `0.475233 ms` per sample—an empirical time ratio (SLSQP / Dykstra) of `0.034891x`.

This outcome directly contradicts an earlier draft's claim that the pure-NumPy Dykstra loop outperformed SLSQP. In hindsight, the speed gap is unsurprising: SLSQP delegates its heavy lifting to compiled Fortran routines with minimal Python overhead, whereas Dykstra's cyclic updates execute hundreds of small array operations inside a Python loop for each cycle. Of course, single-pass timings are sensitive to hardware, BLAS threading, and library versions, so these numbers should be read as an environment-specific observation rather than a universal benchmark.

4.1 Verification Summary

All eight automated tests in our verification suite passed without issue:

1. **Test 1:** The 2D analytical walkthrough (`[4, 5] -> [1.5, 2.5]` on $x_1 + x_2 \leq 4$) from Section 1.1.
2. **Test 2:** Bitwise preservation of already-feasible inputs (`[1, 2]` on $x_1 + x_2 \leq 4$).
3. **Test 3:** Explicit rejection (`ValueError`) of zero, `NaN`, and `Inf` normal vectors, incompatible matrix/vector dimensions, and invalid tolerances or iteration budgets.
4. **Test 4:** Verification of the 10^{-14} squared-norm cutoff (accepting $\|g\|_2 = 10^{-6}$, rejecting $\|g\|_2 = 10^{-8}$) and the reconstructed $+10^{-12}$ denominator offset flaw (`[2, 5] -> [1.5, 5]` instead of `[1, 5]`).
5. **Test 5:** Large-scale 2D normal projection ($x = [4, 3]$ with $g = [10^6, 0]$ and $\beta = 2 \times 10^6$ projecting accurately to `[2, 3]`).
6. **Test 6:** Tight agreement with SciPy's SLSQP solver: when running Dykstra with `step_tol = 1e-12` and `feas_tol = 1e-12`, the L_∞ discrepancy against SLSQP measured `1.152855588771e-12` (well below the explicit `< 1.1e-9` assertion).

7. **Test 7:** Graceful handling of contradictory constraints ($x_1 + x_2 \leq 2$ and $-x_1 - x_2 \leq -5$, yielding signed max violation `3.0` after 50 cycles) without false feasibility or infeasibility-certificate claims.

8. **Test 8:** Explicit separation of iteration-budget exhaustion (`max_iters = 1`, `converged = False`, `is_feasible = False`, `max_violation = 2.32`) from safety certification.

4.2 Limitations

To keep the scope of this work clear, several practical limitations should be noted. All experiments were conducted in standard `float64` precision using synthetic Gaussian vectors and static linear inequalities. We have not evaluated this filter in hardware-in-the-loop setups, real robotic trajectory tracking, nonlinear obstacle avoidance, or closed-loop dynamical systems.

Furthermore, projecting a single output onto a convex polytope only guarantees constraint satisfaction at that instant; in a dynamical system, one-step feasibility does not prevent the system from entering a state where future constraints become impossible to satisfy (lack of recursive feasibility). Finally, as noted in Section 2.3, hitting the maximum iteration cap in Dykstra's algorithm is simply a timeout condition, not a formal proof that the constraint set is empty. Complete execution logs and reproduction scripts are available in the accompanying repository materials.

References

[1] Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.

[2] Dykstra, R. L. (1983). An Algorithm for Restricted Least Squares Regression. *Journal of the American Statistical Association*, 78(384), 837–842.